



National Laboratory of the Rockies

4 — Simple Parametric Study (Scalar Sweep)

This module performs a simple parametric study by sweeping the scalar `GenMix_Target` and plotting the resulting trends.

Learning objectives

- Set up a `ParametricStudy` from baseline SDOM inputs.
- Sweep a scalar parameter with `add_scalar_sweep`.
- Run a multi-case study with explicit output paths.
- Plot and inspect parametric results.

Prerequisites

- You completed modules 1–3.
- You can run SDOM with HiGHS in your environment.

Inputs and scenario used

- Scenario folder: `data/sample_data/`
- Swept scalar: `scalars -> GenMix_Target`
- Time horizon in this training script: `N_HOURS = 7*24*2` (336 time steps)

Reference:

- Parametric analysis: https://natlabrockies.github.io/SDOM/user_guide/parametric_analysis.html

Project/file structure for this module

```
training/4_sdom_parametric_scalar/  
├── README_m4.md  
└── run_m4.py
```

Module suffix and script naming

- Module suffix: `m4`
- Script file: `run_m4.py`

Note on `N_HOURS` in training scripts

`N_HOURS` is used here as a didactic setting for faster turnaround. Real planning runs should use a complete year (`8760` hours).

Step-by-step walkthrough

1. Run:

```
python training/4_sdom_parametric_scalar/run_m4.py
```

2. Review case-level completion messages and final summary.
3. Inspect outputs in:

```
results/training/4_sdom_parametric_scalar/
```

Full runnable script

See [training/4_sdom_parametric_scalar/run_m4.py](#).

Inspect parametric API help from Python

This module introduces `ParametricStudy`, `add_scalar_sweep`, `run`, and `plot_parametric_results`. These are public SDOM APIs for creating a sweep, running the Cartesian set of cases, and plotting cross-case comparisons.

Run these commands from the repository root with the virtual environment active:

```
python -c "from sdom.parametric import ParametricStudy; help(ParametricStudy)"
python -c "from sdom.parametric import ParametricStudy;
help(ParametricStudy.add_scalar_sweep)"
python -c "from sdom.parametric import ParametricStudy; help(ParametricStudy.run)"
python -c "from sdom.analytic_tools import plot_parametric_results;
help(plot_parametric_results)"
```

Expected output excerpts from this environment:

Help on class ParametricStudy in module sdom.parametric.study:

```
class ParametricStudy(builtins.object)
| ParametricStudy(base_data: dict, solver_config: dict, n_hours: int = 8760,
output_dir: Optional[str] = None, n_cores: Optional[int] = None) -> None
|
| Run a multi-dimensional parametric sensitivity study in parallel.
|
| Accepts scalar, storage-factor, and time-series sweep definitions,
| constructs the full Cartesian product of all sweep dimensions, and
| dispatches each combination to a separate worker process via
| :class:`concurrent.futures.ProcessPoolExecutor`.
|
| Parameters
| -----
| base_data : dict
|     SDOM data dictionary returned by :func:`sdom.load_data`.
| solver_config : dict
|     Solver configuration dict from
:func:`sdom.get_default_solver_config_dict`.
| n_hours : int, optional
|     Number of simulation hours. Defaults to ``8760``.
| output_dir : str or None, optional
|     Directory where per-case sub-directories and the summary CSV will be
written.
| n_cores : int or None, optional
|     Number of worker processes.
```

Help on function add_scalar_sweep in module sdom.parametric.study:

```
add_scalar_sweep(self, data_key: str, param_name: str, values: list) -> None
    Register a scalar parameter sweep.
```

Each value in **values** replaces
 ``data[data\_key].loc[param\_name, "Value"]`` for one case dimension.

```
Parameters
-----
data_key : str
```

```

    Key in the SDOM data dict (e.g. ``"scalars"``).
    param_name : str
        Row label of the parameter (e.g. ``"GenMix_Target"``).
    values : list of float
        Discrete values to sweep over.

```

Help on function run in module `sdm.parametric.study`:

```

run(self) -> List[sdm.results.OptimizationResults]
    Execute all parametric combinations in parallel.

```

Constructs the Cartesian product of all registered sweeps, submits every case to a `ProcessPoolExecutor`, reports progress as jobs complete, exports per-case CSVs if `output_dir` was specified, and writes a summary CSV.

Returns

```

-----
list of OptimizationResults
    One entry per combination, in Cartesian-product order.

```

Help on function `plot_parametric_results` in module `sdm.analytic_tools._parametric`:

```

plot_parametric_results(study, results, group_by, hue_by=None, facet_by=None,
output_dir=None, max_cases_per_figure=24, plot_per_case=True) -> None
    Generate sensitivity-analysis plots from a completed ParametricStudy run.

```

Per-case plots are saved under ``<output\_dir>/<case\_name>/plots/``.
 Cross-case comparison plots are saved under
 ``<output\_dir>/sensitivity\_plots/``.

Expected outputs

- One SDOM result per scalar value.
- Exported per-case CSVs and a study summary.
- Parametric plots grouped by `GenMix_Target`.

How to validate results

- Number of returned results equals number of scalar values.
- At least one case is optimal (`termination_condition == "optimal"`).
- Plot files are created in the module output directory.

Troubleshooting

- Multiprocessing issue on Windows: keep the `if __name__ == "__main__":` guard.
- Empty plots: verify at least one successful case.

- Slow runs: reduce number of sweep values for quick tests.

References

- SDOM docs home: <https://natlabrockies.github.io/SDOM/index.html>
- Parametric user guide: https://natlabrockies.github.io/SDOM/user_guide/parametric_analysis.html
- API core: <https://natlabrockies.github.io/SDOM/api/core.html>